

Generative modelling case study

Student name : Indhu Reddy Kottalam Raveendra Reddy

Student id : 24096112

Github repo: <https://github.com/indhu0204/gan-case-study-7PAM2016>

Table of Contents

1. Introduction
2. Part 1 – GANs on Synthetic 2D Data
 - 2.1 Sine-wave GAN reproduction
 - 2.2 New 2D distribution
 - 2.3 Architecture variations and visual comparison
3. Case Study 1 – OCTMNIST Retinal DCGAN
 - 3.1 Data and preprocessing
 - 3.2 DCGAN architecture and training
 - 3.3 Results and limitations
4. Case Study 2 – CICIDS Tabular GAN
 - 4.1 Data and preprocessing
 - 4.2 Tabular GAN and training
 - 4.3 Results and critique
5. Case Study 3 – QuickDraw Birthday Cake DCGAN
 - 5.1 Data and preprocessing
 - 5.2 DCGAN and results
 - 5.3 Quantitative check and reflection
6. Cross-Case Discussion
7. Conclusion
8. References

A . Introduction

Generative adversarial networks (GANs) are deep learning models that synthesise new data via a game between a generator, which produces candidate samples, and a discriminator, which tries to distinguish fake samples from real data. When training succeeds, the generator approximates the underlying data distribution and can be used to create realistic synthetic images, signals or feature vectors for tasks such as data augmentation, privacy-preserving analysis and creative applications.

This report investigates how GANs behave across three contrasting domains using real benchmark datasets. A deep convolutional GAN (DCGAN) is trained on the OCTMNIST subset of MedMNIST to generate retinal optical coherence tomography (OCT) images for four diagnosis categories. A fully connected GAN is applied to tabular network traffic features from the CICIDS 2017 dataset, focusing on Wednesday working-hours BENIGN and DoS/DDoS traffic. A second DCGAN is trained on the Quick, Draw! birthday cake category to produce creative hand-drawn sketches.

Across these case studies, the project aims to implement appropriate GAN architectures for medical images, tabular security data and sketches, to observe how preprocessing and class balance affect training behaviour, and to evaluate realism using loss curves, visual comparisons and lightweight quantitative checks when full metrics such as Fréchet Inception Distance are impractical on CPU hardware. The remainder of the report presents the three case studies and then a cross-case discussion of limitations and future directions.

1. Part 1 – GANs on Synthetic 2D Data

In the first part of the assignment, a simple GAN was implemented in PyTorch to model synthetic one- and two-dimensional data distributions. This served as a warm-up to understand adversarial training behaviour before moving to more complex real datasets. The basic architecture followed the tutorial structure: a multilayer perceptron discriminator receiving low-dimensional inputs and a multilayer perceptron generator mapping latent noise vectors to points in the target distribution.

1.1 Sine-wave GAN reproduction

The initial task was to reproduce a one-dimensional sine-wave GAN. A discriminator was trained to distinguish samples drawn from a fixed sine function from samples produced by a generator, while the generator learned to output points lying along the sine curve rather than random noise. Over training, the generator's outputs moved from scattered values to a band that closely followed the sine wave, illustrating how adversarial training can recover a simple functional relationship in 1D. This experiment reinforced core concepts such as the role of the latent space, the use of binary cross-entropy loss, and the importance of balancing generator and discriminator learning rates.

1.2 New 2D distribution

The second task extended this idea to a two-dimensional synthetic distribution. A noisy parametric curve of the form $y = \sin(2x) + 0.3\cos(5x) + \epsilon$ was chosen, where ϵ is a random noise term. Real samples were created by drawing x from a specified range, computing y from the curve and adding Gaussian noise, resulting in a wiggly band of points in the (x,y) plane. The GAN's generator took 2D latent noise and produced (x,y) points, while the discriminator classified them as real or fake. After training, scatter plots of generated samples showed that the model had learned to follow the overall shape of the parametric curve and its noisy envelope, rather than producing a uniform cloud, demonstrating adversarial learning in a simple 2D setting.

1.3 Architecture variations and visual comparison

Finally, the GAN architecture was modified to explore how design choices affect convergence and sample quality. Experiments varied activation functions (e.g. replacing ReLU with LeakyReLU in the generator), adjusted hidden layer depth and width, and sometimes changed the latent dimensionality. In some configurations the generator converged faster and produced sharper bands of points along the target curve, while in others training became unstable or samples remained diffuse. Comparing scatter plots from the original and modified GANs showed that deeper networks with suitable activations reduced mode dropping and improved coverage of the curve, whereas overly aggressive changes could lead to oscillatory losses or poor approximation.

Overall, Part 1 demonstrated on simple synthetic data that GANs can successfully approximate both 1D and 2D distributions, and that architectural choices have a direct impact on stability and sample quality. These lessons about loss behaviour, balancing generator and discriminator capacity, and reading visual diagnostics informed the design decisions taken in the later case studies on OCTMNIST, CICIDS and QuickDraw.

2. Case Study 1 – OCTMNIST Retinal DCGAN

2.1 Data and preprocessing

The OCTMNIST subset of MedMNIST contains greyscale retinal OCT scans for four diagnosis categories: choroidal neovascularisation, diabetic macular oedema, drusen and normal. It includes 97,477 training images, 10,832 validation images and 1,000 test images, all centred-cropped and resized to $1 \times 28 \times 28$ pixels. Normal scans substantially outnumber disease classes, which affects how easily rare pathologies can be modelled. Example OCTMNIST scans illustrating the four diagnosis categories are shown in Figure 1.

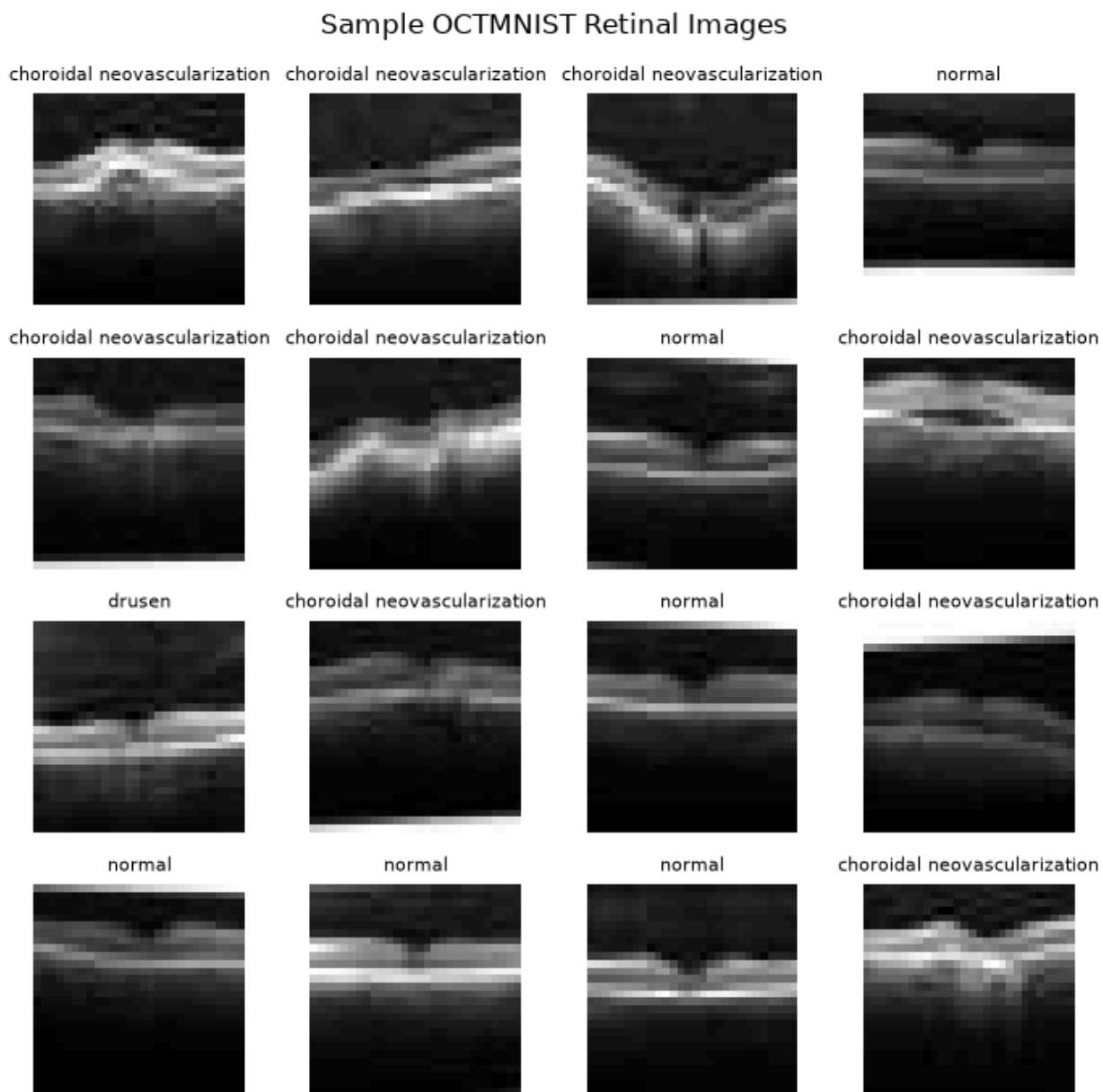


Figure 1: Sample OCTMNIST retinal OCT images from the four diagnosis classes.

The dataset is loaded via the MedMNIST API and transformed with `ToTensor()` followed by `Normalize(mean=[0.5], std=[0.5])` so that intensities lie in $[-1, 1]$ and match a generator with tanh output. OCT images are provided to a PyTorch DataLoader with a fixed batch size and shuffling per epoch, and all experiments run on CPU, constraining training duration.

2.2 DCGAN architecture and training

Retinal OCT structure is modelled using a DCGAN with convolutional generator and discriminator implemented in DCGANGenerator28x28 and DCGANDiscriminator28x28. The generator maps latent noise through transposed convolution layers with batch normalisation and LeakyReLU activations to a $1 \times 28 \times 28$ greyscale image, followed by tanh. The discriminator applies strided convolutions, LeakyReLU and dropout, ending in a single logit for real-versus-fake classification with binary cross-entropy loss.

Training is carried out for three epochs with Adam optimisers and binary cross-entropy loss. On CPU, this requires roughly 30 minutes, so the schedule was deliberately kept short to leave time for the other case studies; longer runs would likely improve fidelity but were not feasible within the assignment's compute budget.

2.3 Results and limitations

Generator and discriminator losses are logged per epoch. The training losses for the OCTMNIST DCGAN over three epochs are shown in Figure 2. Over three epochs, discriminator loss decreases while generator loss increases, indicating that the discriminator rapidly learns to distinguish real from synthetic OCT images, while the generator struggles under the short training regime. Loss curves remain stable and do not show collapse.

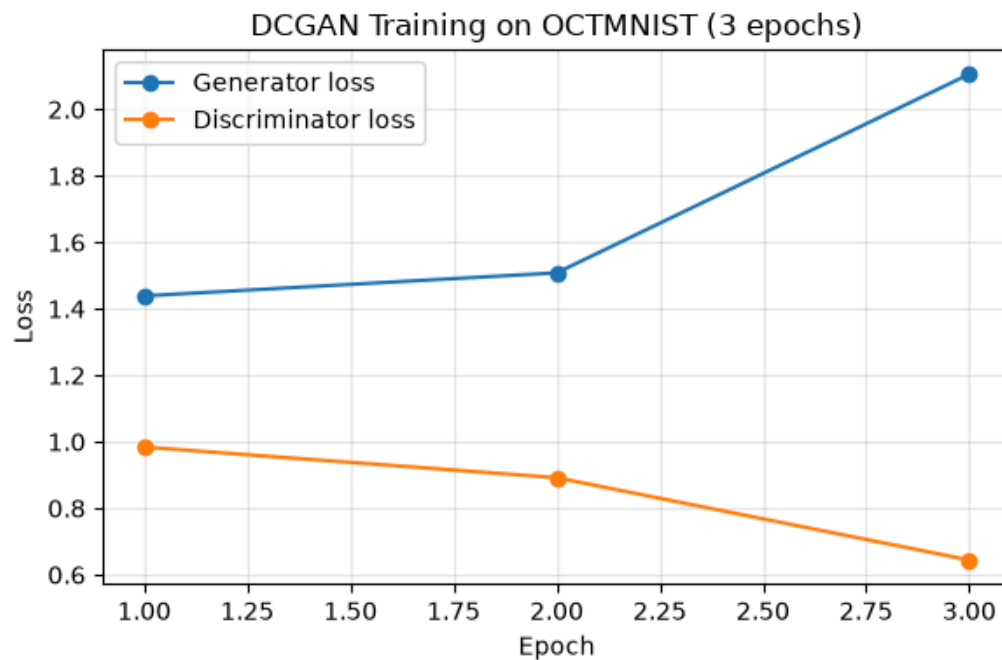


Figure 2: Generator and discriminator loss curves for the OCTMNIST DCGAN over three CPU-trained epochs.

Grid visualisations show that real OCTMNIST scans have layered retinal structures and subtle intensity differences, whereas DCGAN-generated images reproduce the broad layered geometry but often resemble normal scans and rarely show clear pathological features. This reflects both the short training and the underlying class imbalance. Only simple pixel-intensity statistics are compared between real and fake batches due to CPU constraints and the mismatch between standard image-feature metrics and OCT data; more rigorous evaluation would require retinal-specific feature extractors and significantly more compute. Overall, the OCTMNIST experiment shows that a DCGAN can begin to learn OCT structure within tight constraints but highlights the need for longer training, class-conditional generation and domain-appropriate quantitative metrics for high-fidelity medical synthesis.

3. Case Study 2 – CICIDS Tabular GAN

3.1 Data and preprocessing

CICIDS 2017 is a benchmark for intrusion detection containing labelled network flows across several days. Here, the Wednesday working-hours file Wednesday-workingHours.pcap_ISCX.csv is used, focusing on BENIGN flows and DoS/DDoS attacks (Hulk, GoldenEye, slowloris, Slowhttptest, Heartbleed). The file has 692,703 rows and 79 columns, with BENIGN and DoS Hulk dominating and some attacks, such as Heartbleed, occurring only a handful of times.

The notebook filters rows to these labels, strips column names to harmonise "Label" and "Label", and standardises the label column. Infinities are replaced with NaN and rows with missing numeric data are dropped, leaving 691,406 flows. The feature matrix X is formed from 78 numeric columns, and features are standardized with StandardScaler to zero mean and unit variance. The scaled matrix is wrapped in a TensorDataset and DataLoader (batch size 256, shuffling), giving 2,701 batches per epoch.

3.2 Tabular GAN and training

Because CICIDS flows are feature vectors, a fully connected GAN is used. The generator TabularGenerator takes a 32-dimensional latent vector and passes it through dense layers with nonlinear activations to produce 78-dimensional outputs. The discriminator TabularDiscriminator receives 78-dimensional inputs and outputs a single logit; both networks operate in standardised feature space without label conditioning or explicit constraints.

Training runs for ten epochs with binary cross-entropy loss and Adam optimisers ($\text{lr} = 2 \times 10^{-4}$). Batches of real samples are drawn from the DataLoader, and matching synthetic samples are generated from latent noise. Losses are recorded at each epoch for later analysis.

3.3 Results and critique

Loss curves show generator loss decreasing from roughly 1.99 to around 1.7 over ten epochs, while discriminator loss increases slightly and stabilises near 0.63–0.65, suggesting moderate learning without obvious collapse. Figure 3 visualises generator and discriminator losses during CICIDS GAN training.

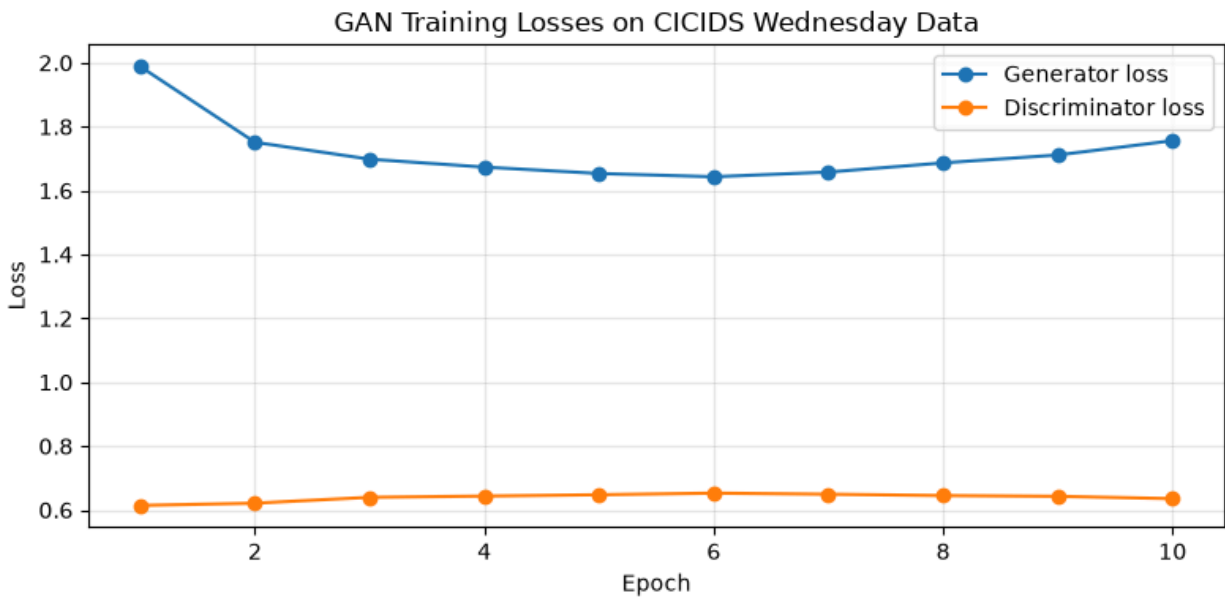


Figure 3: GAN training losses on CICIDS Wednesday data. Generator loss decreases while discriminator loss stabilises near 0.63–0.65.

A PCA projection of 5,000 real and 5,000 fake samples reveals that synthetic points occupy the same dense central region as real traffic near the origin of the first principal component, but they rarely reach the extreme outlier regions that real flows occupy, implying that the GAN captures typical behaviour but not large or unusual flows. The PCA projection comparing real and GAN-generated CICIDS feature vectors is shown in Figure 4.

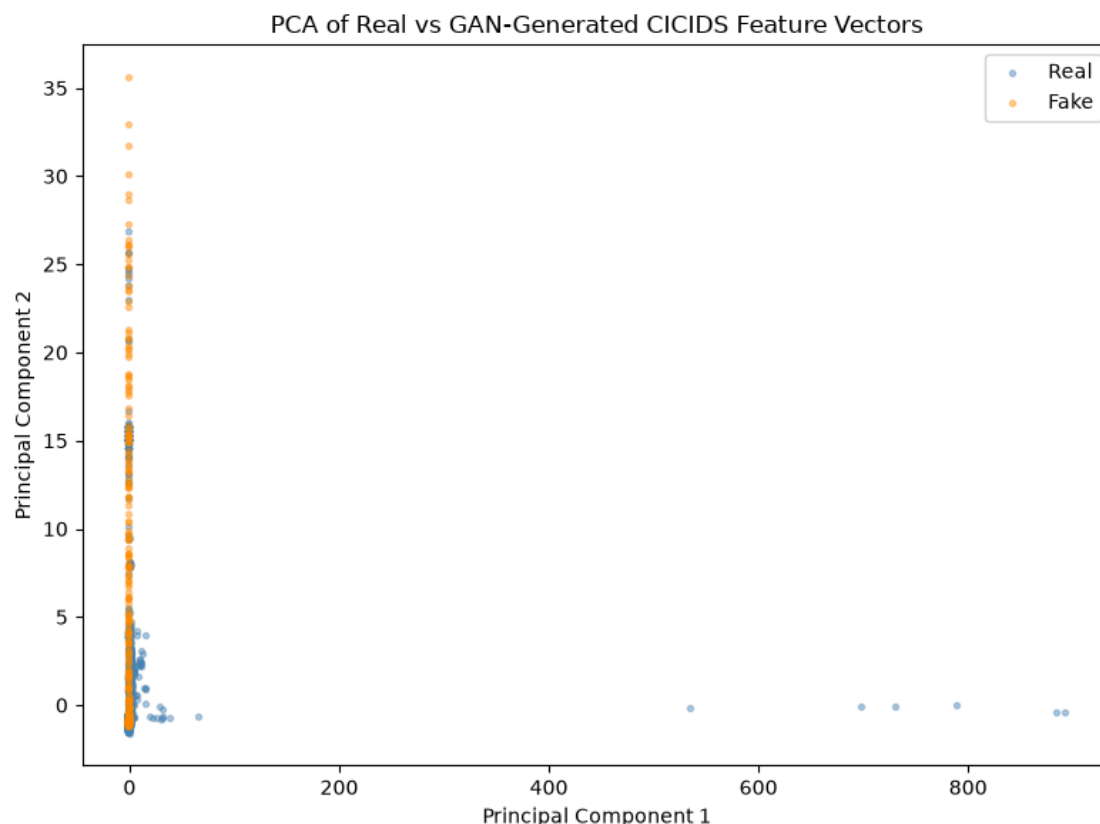


Figure 4: PCA of 5,000 real and 5,000 GAN-generated CICIDS feature vectors. Synthetic samples cluster in the central region but rarely reach extreme outliers along the first principal component.

To examine feature-level behaviour, the notebook compares raw means and standard deviations for several key features in 5,000 real versus 5,000 synthetic flows, after inverting the scaler. For Flow Duration, the real mean is approximately 2.9×10^7 with a standard deviation of 4.3×10^7 ; synthetic flows have a mean around 1.7×10^6 and much smaller spread; similar mismatches appear for packet counts and byte totals. In some cases, the generator produces negative values for count-like features, which are impossible in real traffic and show that basic semantic constraints are not enforced. These findings support the conclusion that the model mainly reproduces the dense central region in transformed space but fails to match the true distribution in raw feature space, especially for extreme flows.

As a result, the CICIDS experiment indicates that, with the current architecture and training regime, the tabular GAN does not produce synthetic traffic that can realistically replace the original data. Prior work on CICIDS stresses that careful preprocessing—feature selection, log transforms and robust scaling—is essential for good performance, and naive GAN training on raw features is unlikely to capture such skewed distributions. A more ambitious extension would combine multiple days, harmonise preprocessing and adopt a conditional or constrained generative model that respects non-negativity and attack labels.

4. Case Study 3 – QuickDraw Birthday Cake DCGAN

4.1 Data and preprocessing

The Quick Draw! The dataset contains many sketch categories; here the birthday cake class is used. A local `birthday_cake.npy` file provides 144,982 sketches as 784-dimensional vectors, each corresponding to a $28 \times 28 \times 28 \times 28$ bitmap when reshaped, with white strokes on a black background. The notebook loads this file, confirms shape and dtype, reshapes to $(144982, 28, 28)$ and visualises sample grids showing varied cake designs. Representative QuickDraw birthday cake sketches are shown in Figure 5.



Figure 5: Sample QuickDraw birthday cake sketches with varied hand-drawn designs.

For DCGAN training, sketches are cast to float32, normalised to $[-1, 1]$ via $(\text{sketches} / 127.5) - 1.0$, and expanded to $(N, 28, 28, 1)$. A `tf.data.Dataset` is built from these tensors, shuffled and batched with `BATCH_SIZE = 128`, providing a stream of greyscale birthday-cake sketches for adversarial training.

4.2 DCGAN and results

The TensorFlow DCGAN uses a generator that maps a 100-dimensional latent vector through a dense layer to a $7 \times 7 \times 256$ feature map, then applies transposed convolutions with batch normalisation and LeakyReLU activations to reach $28 \times 28 \times 1$ outputs with tanh. The discriminator applies strided convolutions, LeakyReLU and dropout, flattens and outputs a single logit; both networks are trained with binary cross-entropy loss from logits.

Training for five epochs on CPU with Adam ($\text{lr} = 1 \times 10^{-4}$) yields stable losses and progressively clearer cake-like outputs. Early epochs produce noisy blobs with limited structure; by epoch 4, generated images show distinct cake bases and vertical candle strokes. A real-fake panel comparing eight real sketches with eight DCGAN outputs shows that the model captures the main cake-and-candles motif but produces softer outlines and less stylistic diversity than human drawings, reflecting both the short training and lack of regularisation for variety. The real-fake comparison is shown in Figure 6

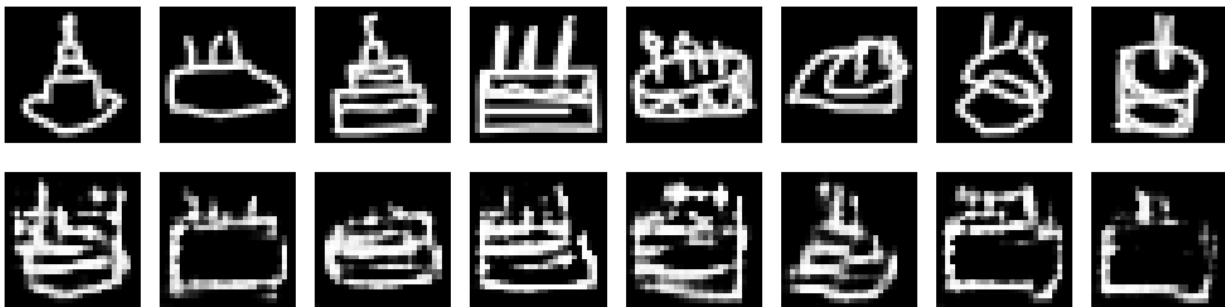


Figure 6: Real QuickDraw birthday cakes (top row) and DCGAN-generated cakes (bottom row) after five training epochs.

4.3 Quantitative check and reflection

As a simple quantitative check, mean and standard deviation of pixel intensities are computed for 1,000 real and 1,000 synthetic sketches, after rescaling from $[-1, 1]$ to $[0, 1]$. Real images have mean 0.198 and standard deviation 0.344, while generated images have mean 0.204 and standard deviation 0.342, showing that the DCGAN has learned very similar brightness and contrast statistics to the real data. This coarse agreement supports the visual impression of plausible cakes against dark backgrounds, although it does not measure higher-level aspects such as stroke style or diversity.

A full Fréchet Inception Distance evaluation was not attempted due to CPU constraints and the mismatch between Inception features and low-resolution line drawings. Overall, the QuickDraw experiment demonstrates that a DCGAN can, within five CPU-trained epochs, generate synthetic birthday-cake sketches that are visually and statistically close to real ones, while leaving clear room for improvement via longer training, architectural tweaks or multi-class sketch modelling and sketch-specific evaluation metrics.

5. Cross-Case Discussion

The three case studies show that GAN performance is strongly shaped by the data domain. In OCTMNIST and QuickDraw, images are low resolution and naturally suited to convolutional DCGANs, which quickly learn layered retinal structure and cake-and-candles shapes even with few CPU epochs. In CICIDS, by contrast, flows are high-dimensional, heavy-tailed feature vectors with strict semantic constraints, and a simple fully connected GAN struggles to match the true distribution, particularly for extreme and rare behaviours.

Evaluation strategies also differ by domain. For OCTMNIST and QuickDraw, humans can visually assess realism, and simple pixel statistics provide supporting evidence when heavier metrics are impractical. For CICIDS, realism must be judged via feature distributions: PCA structure and per-feature statistics reveal large mismatches and invalid values despite stable losses, showing that adversarial loss alone is not a reliable indicator of realistic synthetic data. All experiments were constrained to short runs on CPU (three epochs for OCTMNIST, ten for CICIDS, five for QuickDraw), which sufficed for image-based cases but likely contributed to underfitting in the tabular case.

Across the notebooks, preprocessing and inductive bias emerge as critical factors. Normalisation to $[-1,1]$ aligns naturally with DCGANs for images, while standard scaling alone is insufficient for skewed traffic features, allowing semantics to be violated. Future improvements therefore need to be domain-specific: conditional DCGANs and retinal feature metrics for OCT, sketch-aware models and metrics for QuickDraw, and constrained or conditional tabular generative models for CICIDS.

6. Conclusion

The project implemented and evaluated GANs across retinal OCT images, tabular cybersecurity traffic and hand-drawn sketches. The OCTMNIST and QuickDraw experiments demonstrate that convolutional GANs can learn the broad structure of image data under modest CPU budgets, generating plausible retinal scans and recognisable birthday-cake sketches within only a few epochs. These results illustrate the practical value of DCGANs for low-resolution image synthesis in both scientific and creative settings.

The CICIDS experiment shows that generative modelling becomes more challenging for high-dimensional, skewed and constrained tabular data. Although the GAN learned part of the

central feature distribution and produced overlapping PCA structure with real flows, feature-wise comparisons revealed that it failed to reproduce the correct scale, variability and validity of important traffic characteristics. Realistic tabular synthesis therefore requires careful preprocessing, domain constraints and more specialised models than a basic fully connected GAN.

A wider conclusion is that evaluation strategy matters as much as generation quality. Visual comparison, loss curves and simple statistics were sufficient to provide evidence under CPU-only constraints but also exposed the limits of lightweight evaluation. More rigorous future work would incorporate domain-specific metrics, conditional generation, longer training schedules and improved architectures tailored to each modality. Viewed together, the three case studies offer a practical demonstration of GAN behaviour across very different problem settings and highlight both their potential and their sensitivity to data representation, imbalance and computational budget.

For the datasets and core GAN background, you can mirror the wording and hyperlinks given in the task sheet, but formatted as a proper References section. Below is a clean section you can paste at the end of your report.

References

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y. (2014) 'Generative Adversarial Nets', *Advances in Neural Information Processing Systems*, 27. Available at: <https://arxiv.org/abs/1406.2661>.

Yang, J., Shi, R., Wei, D., Liu, Z., Zhao, L., Ke, B., Zhang, J., Wang, Q., Li, L., Zhang, Y., Sun, M., Liu, Q., Zhang, J. and Zhang, Y. (2021) 'MedMNIST v2: A large-scale lightweight benchmark for 2D and 3D biomedical image classification', *arXiv preprint arXiv:2110.14795*. MedMNIST datasets available at: <https://medmnist.com>.

Canadian Institute for Cybersecurity (2017) *Intrusion Detection Evaluation Dataset (CIC-IDS2017)*. University of New Brunswick. Available at: <https://www.unb.ca/cic/datasets/ids-2017.html>.

Google Creative Lab (2018) 'Drawings in the Cloud: Introducing the Quick, Draw! dataset', Google Cloud Blog. Available at: <https://cloud.google.com/blog/products/gcp/drawings-in-the-cloud-introducing-the-quick-draw-dataset>.

Google Creative Lab (2017) *Quick Draw Dataset*. Public archive of 50 million sketches across 345 categories. Available at: <https://github.com/googlecreativelab/quickdraw-dataset>.